



Malware Analysis Nro o (Zero) Eugenio Deluca (KarasuR00T) Date - 15/03/2023

Environment (VirtualBox Lab)

Rever (BEGIN) [Corriendo] - Oracle VM VirtualBox

Archivo Máquina Ver Entrada Dispositivos Ayuda

Configuración

Inicio

Buscar una opción de configuración

Sistema

- Pantalla
- Sonido
- Notificaciones y acciones
- Asistente de concentración
- Energía y suspensión
- Almacenamiento
- Tableta
- Multitareas
- Proyectar en este equipo

Acerca de

Nombre del dispositivo

Procesador	Intel(R) Core(TM) i3-7100 CPU @ 3.90GHz 3.91 GHz
RAM instalada	3,91 GB
Id. del dispositivo	
Id. del producto	
Tipo de sistema	Sistema operativo de 64 bits, procesador x64
Lápiz y entrada táctil	La entrada táctil o manuscrita no está disponible para esta pantalla

Copiar

Cambiar el nombre de este equipo

Especificaciones de Windows

Edición	Windows 10 Enterprise
Versión	22H2
Se instaló el	22/03/2023
Compilación del SO	19C
Experiencia	Win 12C

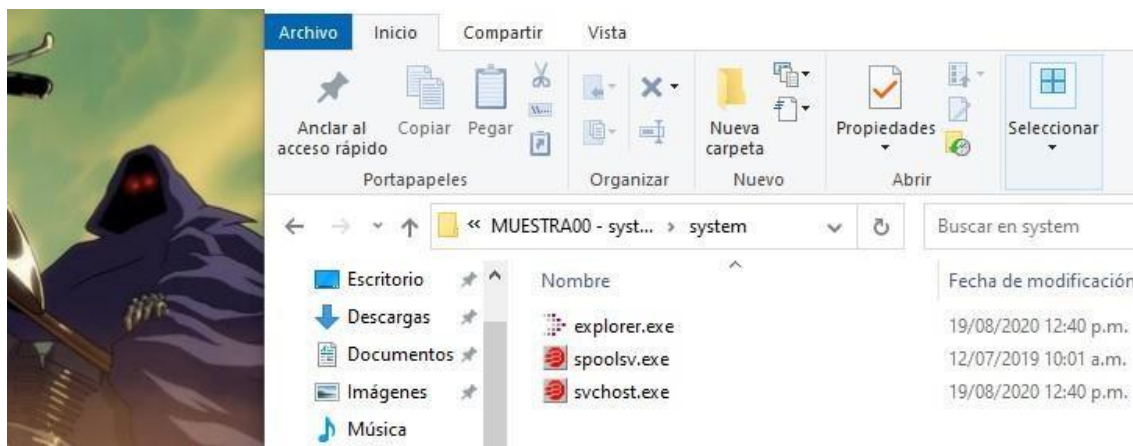
Sample:

MD5: 9c81dc510fdd597670e98687aa528981

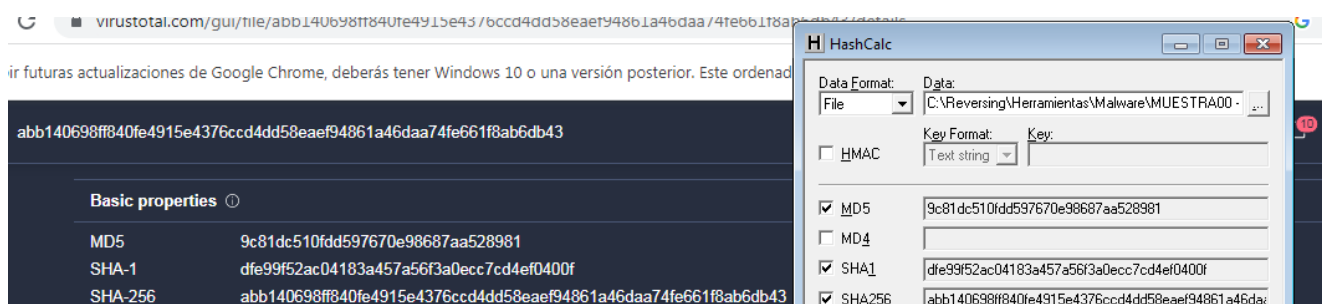
SHA1: dfe99f52ac04183a457a56f3a0ecc7cd4ef0400f

SHA-256: abb140698ff840fe4915e4376ccd4dd58aef94861a46daa74fe661f8ab6db43

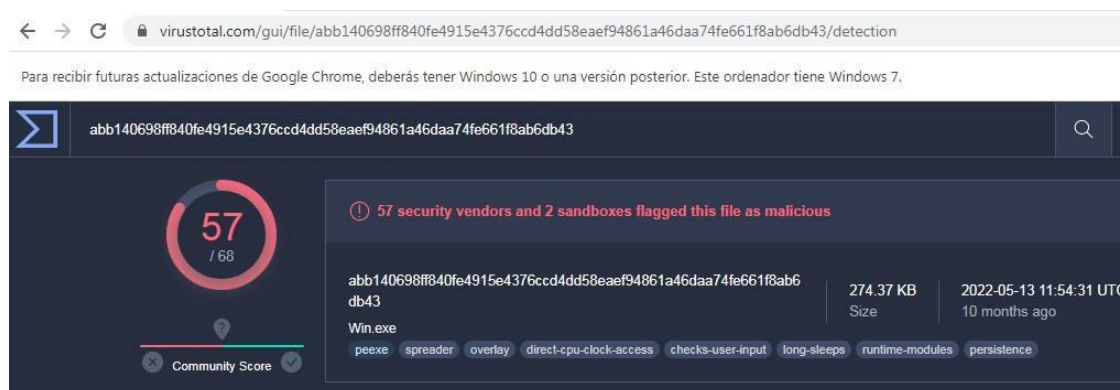
The reason for this analysis stems from detecting it on a client's PC; I became curious to understand what it does using the knowledge I currently possess. While there are three files, I will focus my analysis solely on explorer.exe, as it is the one that caught my attention the most.



First step: upload the suspicious file to VirusTotal. Additionally, use the 'HashCalc' tool to check its hashes (MD5, SHA-1, and SHA-256 only).



On VirusTotal, this file is flagged as malicious, with a community score of 57 out of 68 (along with detections from several solutions, including two sandboxes).



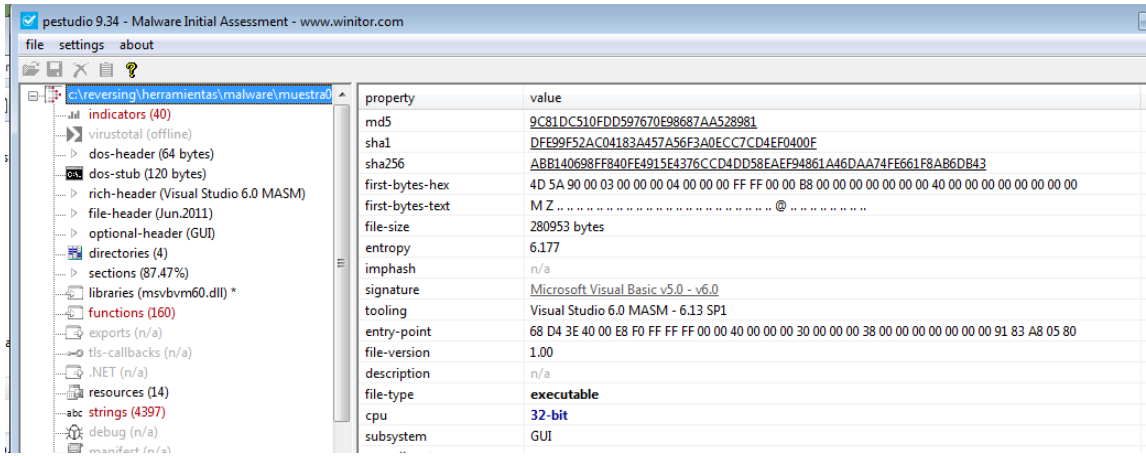
According to the page, this is the history:

Creation Time	2011-06-14 19:01:16 UTC
First Submission	2022-05-13 11:54:31 UTC
Last Submission	2022-05-13 11:54:31 UTC
Last Analysis	2022-05-13 11:54:31 UTC

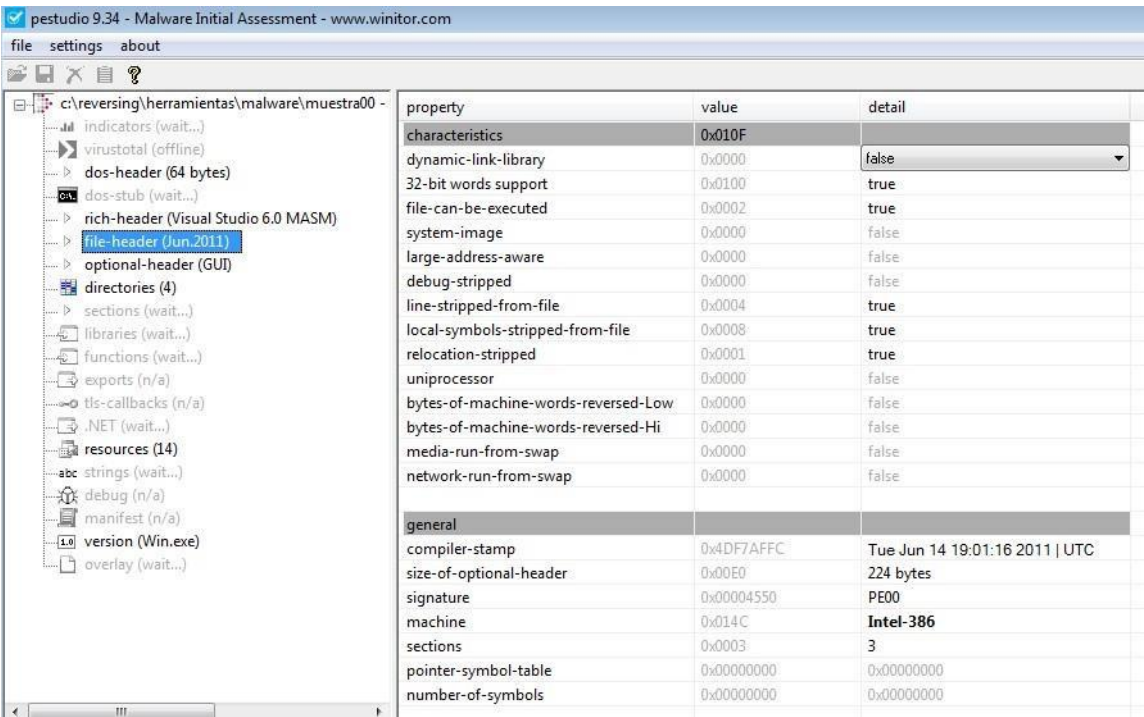
It is identified as explorer.exe – Win – Win.exe. Additionally, it reports the sections it contains.

Header						
Target Machine	Intel 386 or later processors and compatible processors					
Compilation Timestamp	2011-06-14 19:01:16 UTC					
Entry Point	13936					
Contained Sections	3					
Sections						
Name	Virtual Address	Virtual Size	Raw Size	Entropy	MD5	Chi2
.text	4096	173864	176128	5.95	6805a78c569a1d85aabd684522fa4102	2565414.75
.data	180224	7028	4096	0	620f0b67a91f7f74151bc5be745b7110	1044480
.rsrc	188416	63036	65536	5.55	66b64e0769805b16dbf4becf247e6d7f	2222195

Before proceeding, as a security measure, I am removing the network adapter and creating a restore point just in case. I am also renaming explorer.exe to explorer.exe_ to avoid accidental execution. If I open the executable in PEstudio, we can see some interesting data:

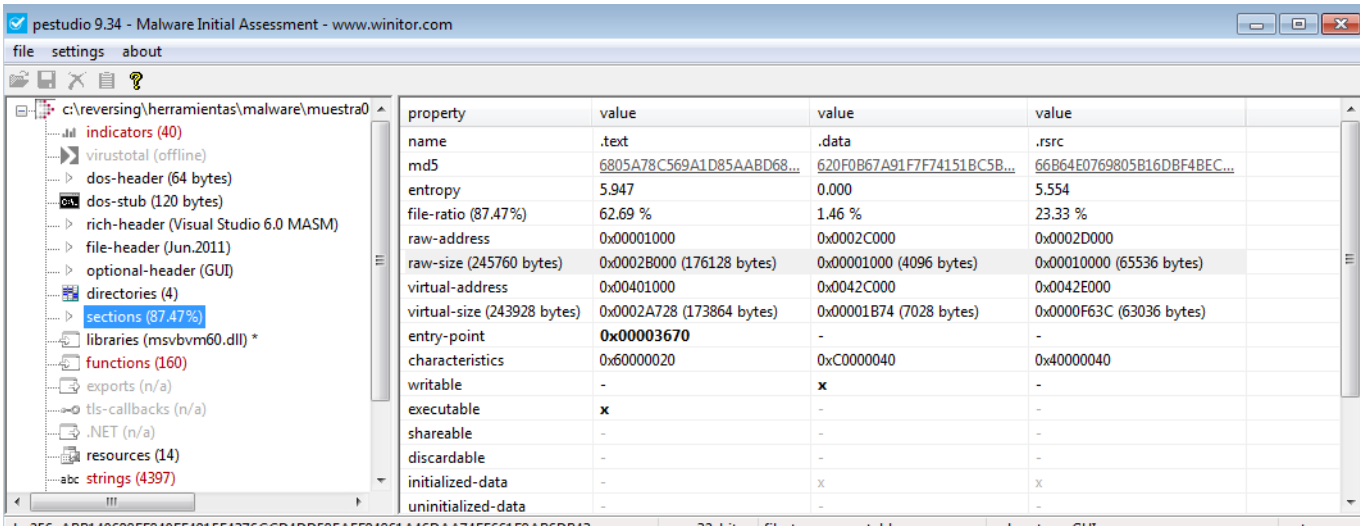


We can see that it is indeed a 32-bit executable, apparently developed in Visual Studio 6.0. It is worth noting that it displays Microsoft Visual Basic v5.0 – v6.0.



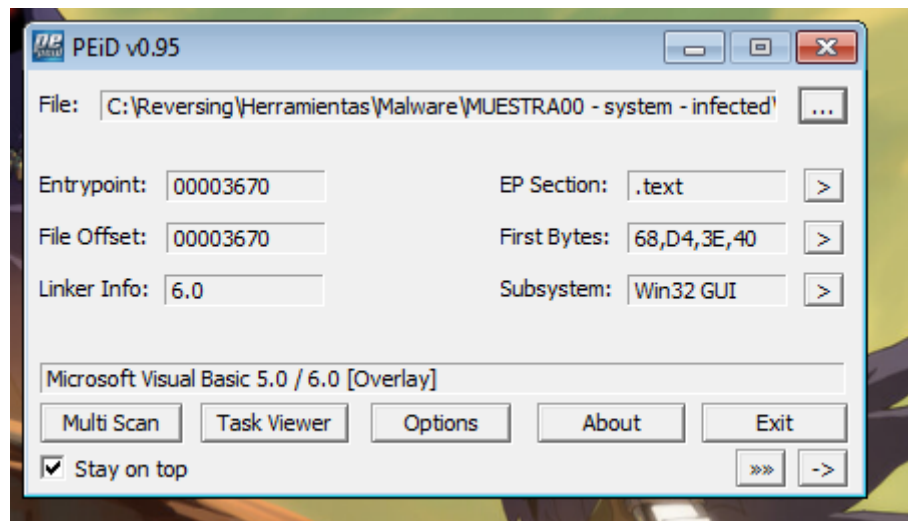
property	value	detail
characteristics	0x010F	
dynamic-link-library	0x0000	false
32-bit words support	0x0100	true
file-can-be-executed	0x0002	true
system-image	0x0000	false
large-address-aware	0x0000	false
debug-stripped	0x0000	false
line-stripped-from-file	0x0004	true
local-symbols-stripped-from-file	0x0008	true
relocation-stripped	0x0001	true
uniprocessor	0x0000	false
bytes-of-machine-words-reversed-Low	0x0000	false
bytes-of-machine-words-reversed-Hi	0x0000	false
media-run-from-swap	0x0000	false
network-run-from-swap	0x0000	false
general		
compiler-stamp	0x4DF7AFC	Tue Jun 14 19:01:16 2011 UTC
size-of-optional-header	0x00E0	224 bytes
signature	0x00004550	PE00
machine	0x014C	Intel-386
sections	0x0003	3
pointer-symbol-table	0x00000000	0x00000000
number-of-symbols	0x00000000	0x00000000

Here we can see that it has no dynamic linking, and it obviously supports 32-bit. It is confirmed once again that it was compiled on 06/14/2011 and contains 3 sections.

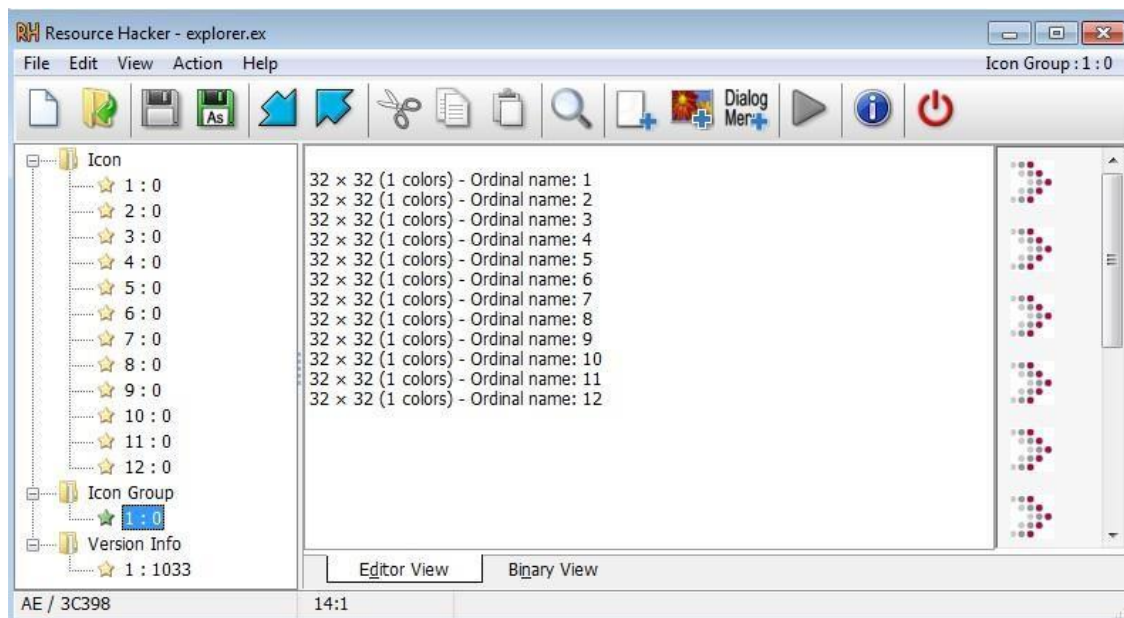


property	value	value	value
name	.text	.data	.rsrc
md5	6805A78C569A1D85AABD68...	620F0B67A91F7F74151BC5B...	66B64E0769805816DBF4BEC...
entropy	5.947	0.000	5.554
file-ratio (87.47%)	62.69 %	1.46 %	23.33 %
raw-address	0x00001000	0x0002C000	0x0002D000
raw-size (245760 bytes)	0x0002B000 (176128 bytes)	0x00001000 (4096 bytes)	0x00010000 (65536 bytes)
virtual-address	0x00401000	0x0042C000	0x0042E000
virtual-size (243928 bytes)	0x0002A728 (173864 bytes)	0x00001B74 (7028 bytes)	0x0000F63C (63036 bytes)
entry-point	0x00003670	-	-
characteristics	0x00000020	0xC0000040	0x40000040
writable	-	x	-
executable	x	-	-
shareable	-	-	-
discardable	-	-	-
initialized-data	-	x	x
uninitialized-data	-	-	-

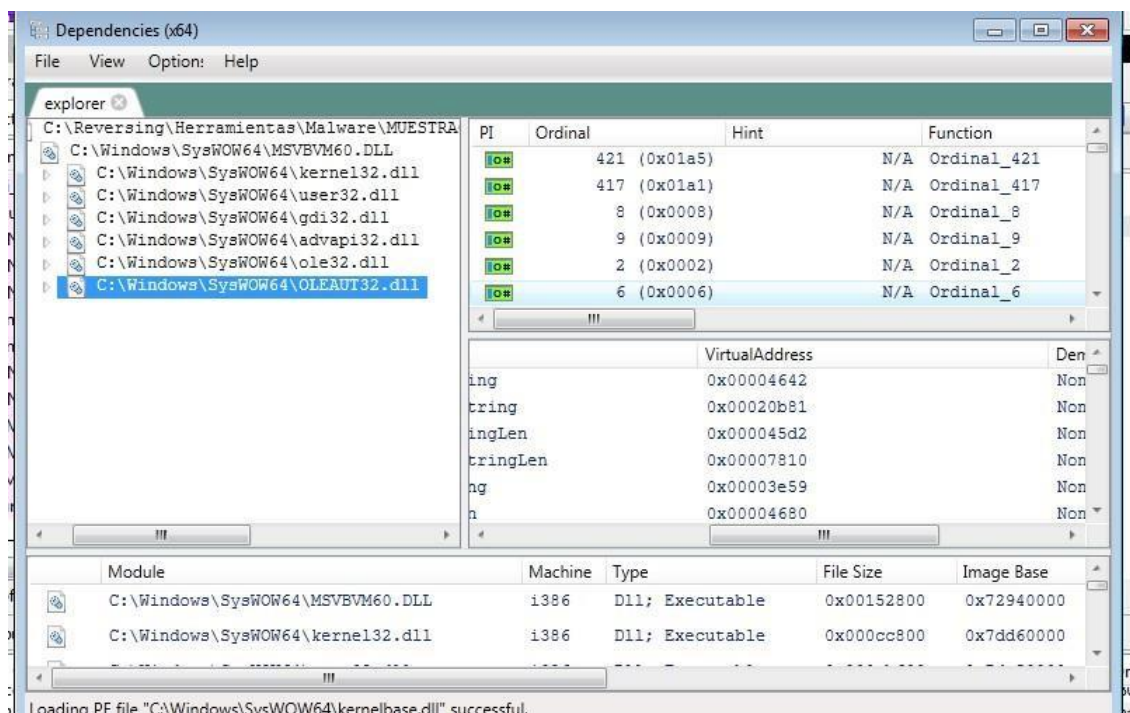
We can see that it has execution and write permissions, and at first glance, it does not appear to be packed. Additionally, it imports the msvbm60.dll library; roughly speaking, this means it is a module for the Microsoft Visual Basic virtual machine. Using PEiD, we verify that it does not seem to be packed, confirming it is an executable compiled in Visual Basic.



Resource Hacker did not return any data of interest. It only shows the executable's icon.



In Dependencies, it indicates that it has all of those DLLs (I apologize, but I will be searching for most of this information online, as I do not have prior experience):



GDI32.dll -> Graphics Device Interface (GDI) functions for device output, such as drawing and font management.

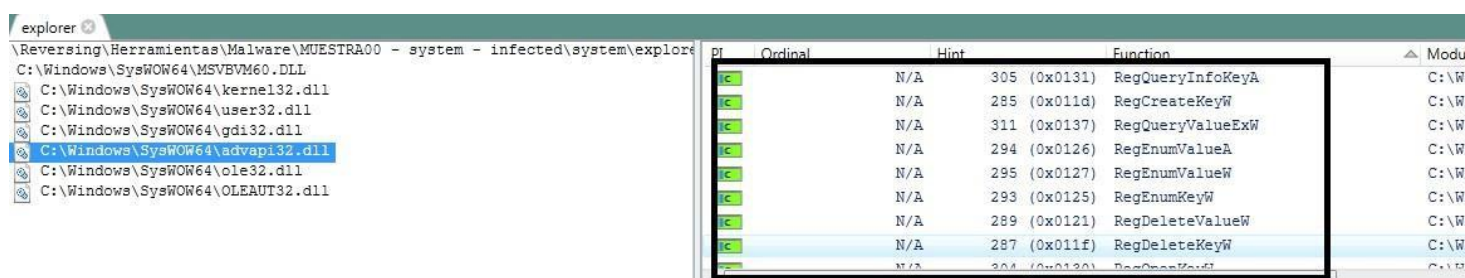
Kernel32.dll -> Low-level operating system functions for memory management and resource handling.

User32.dll -> Windows management functions for handling messages, timers, menus, and communications. (Nota: En Windows la DLL se llama User32.dll, no "Usuario32").

Ole32.dll -> A library that contains core Object Linking and Embedding (OLE) functions.

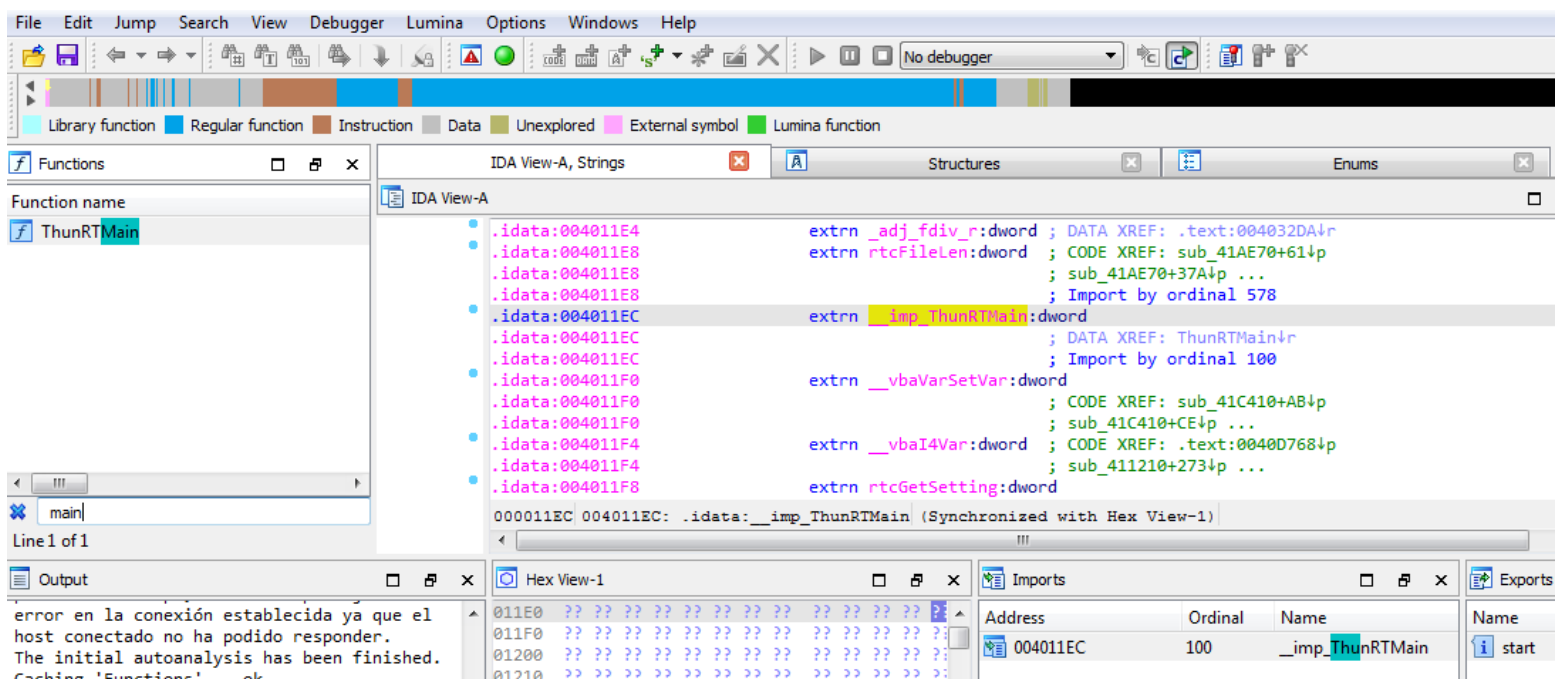
Oleaut32.dll -> A core system library responsible for Object Linking and Embedding (OLE) Automation. It provides essential services for cross-component communication and data manipulation, commonly utilized by Visual Basic applications.

Advapi32.dll -> Part of the Advanced API Services library, which handles numerous system functions, including security calls and Windows Registry management. It is a critical operating system process required for the PC to function correctly.



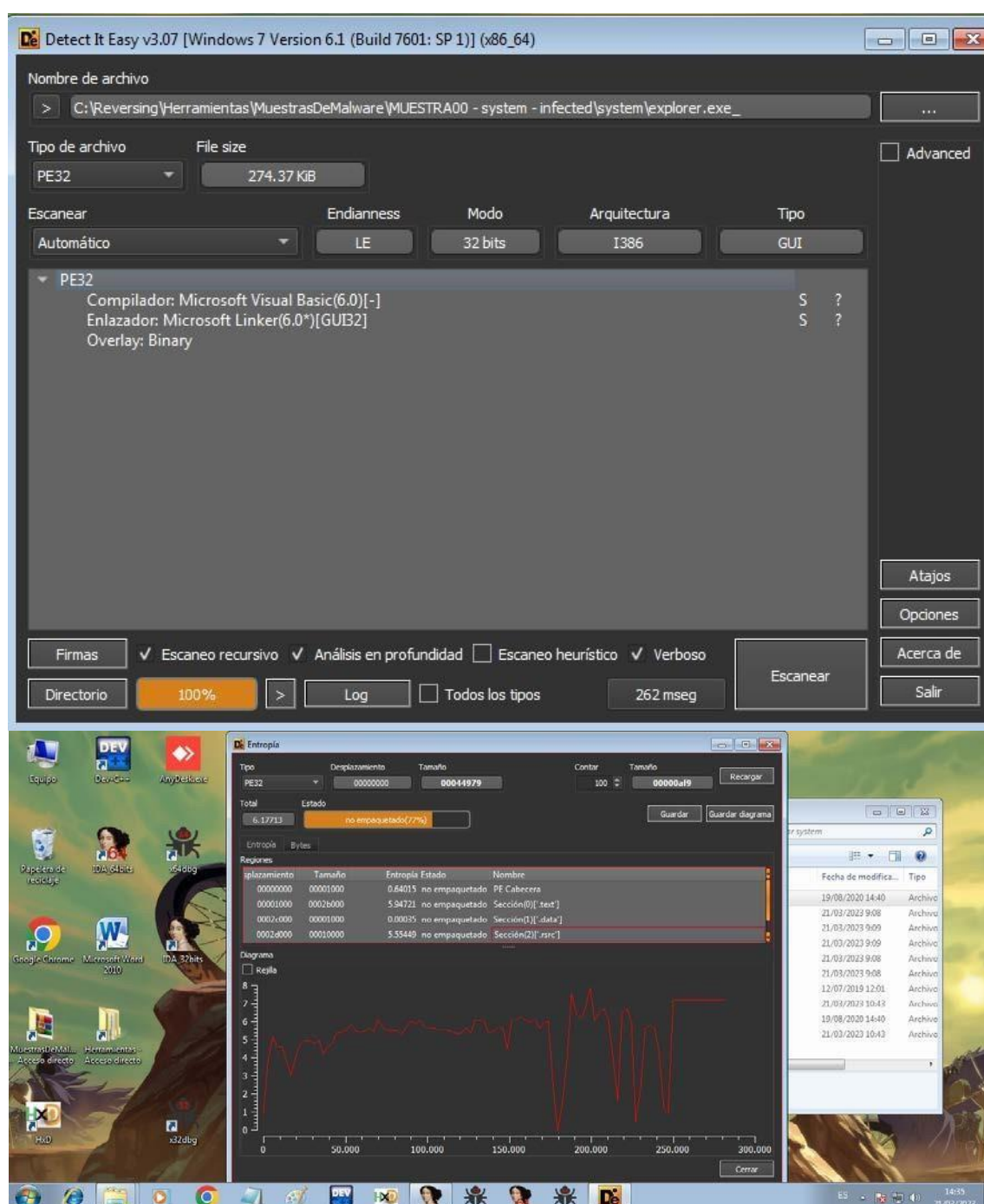
This caught my attention: it features functions that allow for creating registry keys, deleting them, and other actions... It could be nothing, but there is definitely something suspicious here.

At this point, I decided to drop the sample into IDA Pro.



There, I searched for the function containing 'main' and found this in the imports (it is not visible in the image, but it calls msvbm60.dll, and the function is named ThunRTMain -> the imported main comes from a system DLL: MSVBVM60.dll).

To clear any doubts, I dropped the executable into DIE (Detect It Easy), and it does not detect any packer.



The following 'suspicious' functions are found within IDA; it also features image-related functions such as `GdipCreateBitmapFromHBITMAP` and `GdipSaveImageToFile`:

SetWindowsHookExA: Installs an application-defined hook procedure into a hook chain to monitor the system for certain types of events. These events are associated either with a specific thread or with all threads in the same desktop as the calling thread. One of the many parameters it accepts, for instance, is WH_KEYBOARD_LL (installs a hook procedure that monitors low-level keyboard input events).

GetForegroundWindow

GetWindowThreadProcessId: Retrieves the identifier of the thread that created the specified window and, optionally, the identifier of the process that created the window.

CreateToolhelp32Snapshot: Takes a snapshot of the specified processes, as well as the heaps, modules, and threads used by these processes.

GetModuleFileNameExA: Retrieves the fully qualified path for the file containing the specified module.

GetCurrentProcess: Retrieves a pseudo-handle for the current process.

AdjustTokenPrivileges: Enables or disables privileges in the specified access token.

GetExitCodeProcess: Retrieves the termination status of the specified process.

SHGetSpecialFolderLocation // SHGetFolderLocation:

[SHGetSpecialFolderLocation is not supported and may be altered or unavailable in the future. Instead, use SHGetFolderLocation.] – Retrieves a pointer to the ITEMIDLIST structure of a special folder.

GetFileAttributesA: Retrieves file system attributes for a specified file or directory. (Additionally, GetFileAttributesEx. To perform this operation as a transacted operation, use the GetFileAttributesTransacted function).

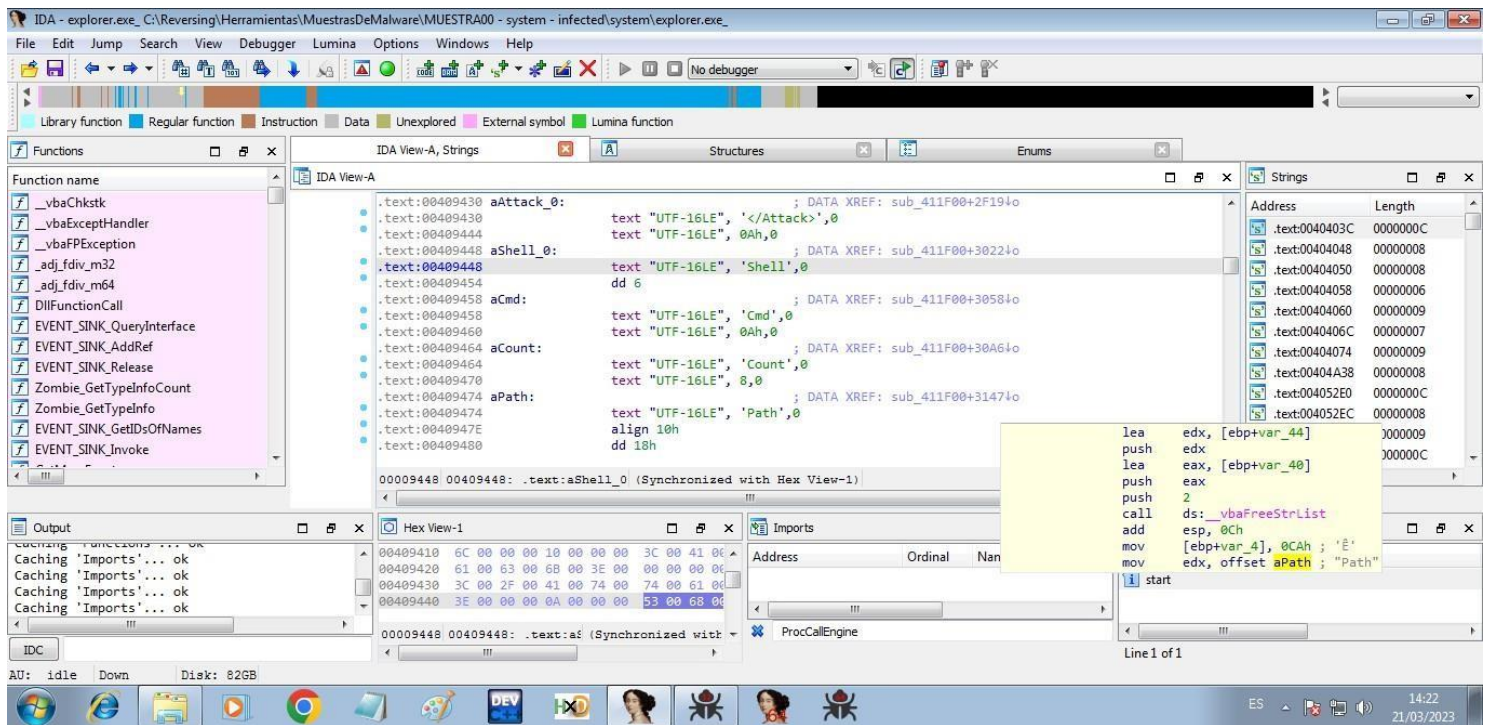
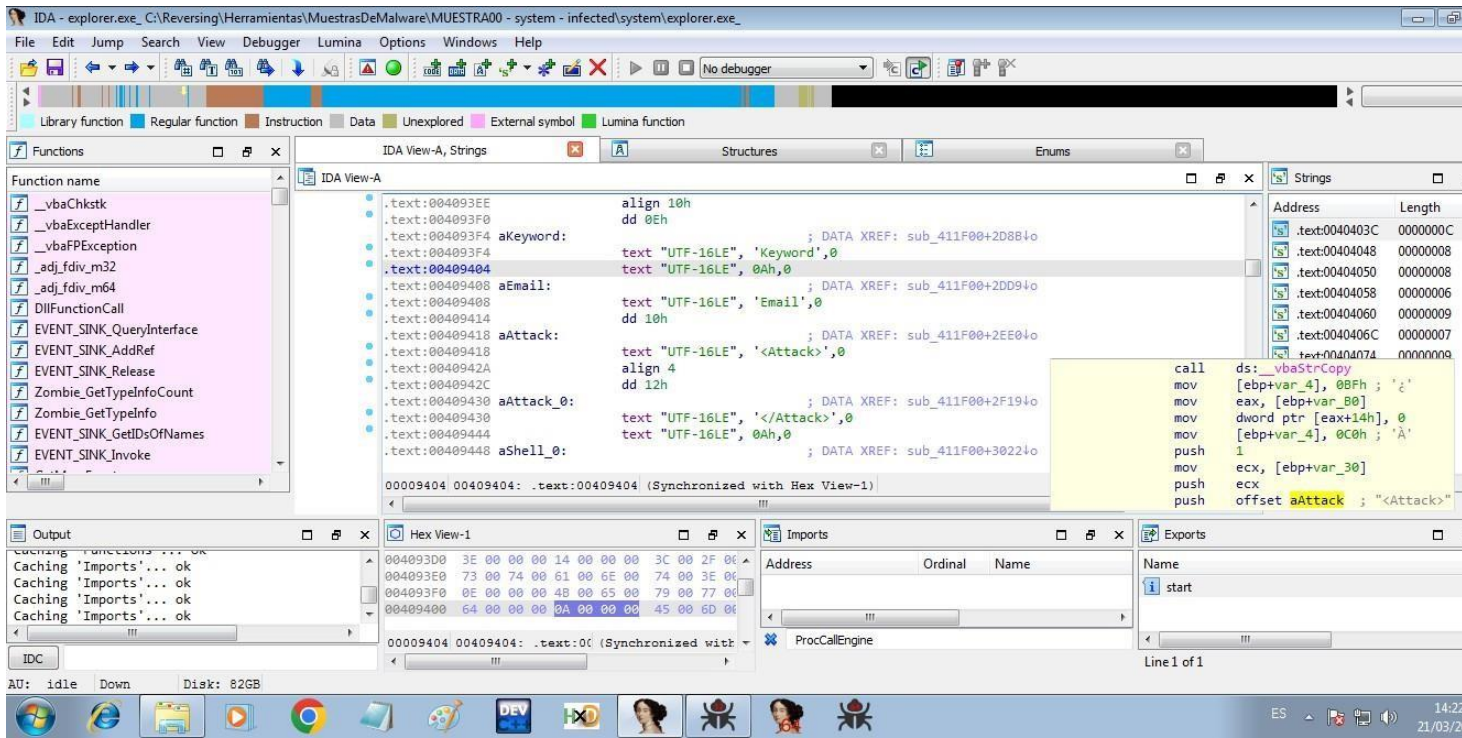
WritePrivateProfileStringA: Copies a string into the specified section of an initialization file (.ini file).

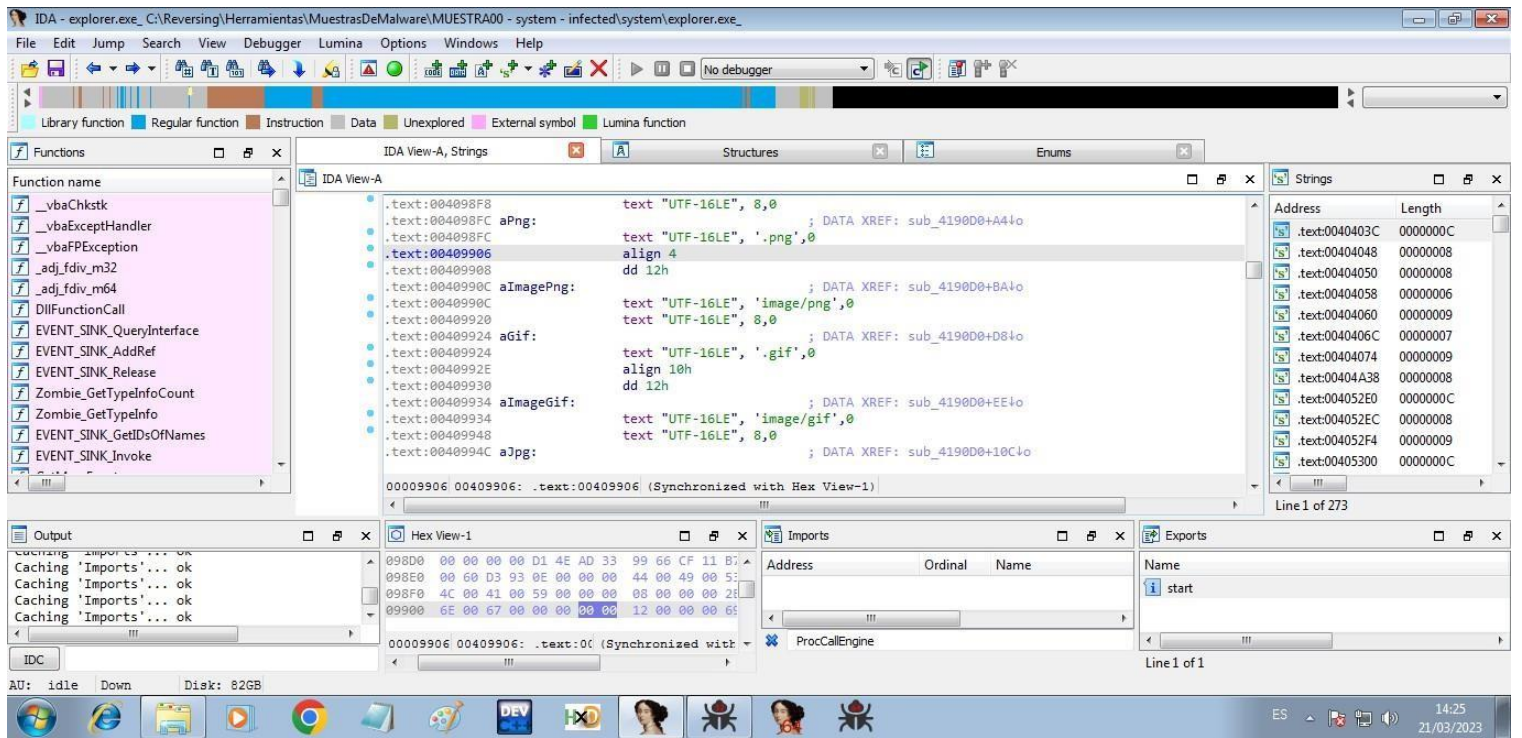
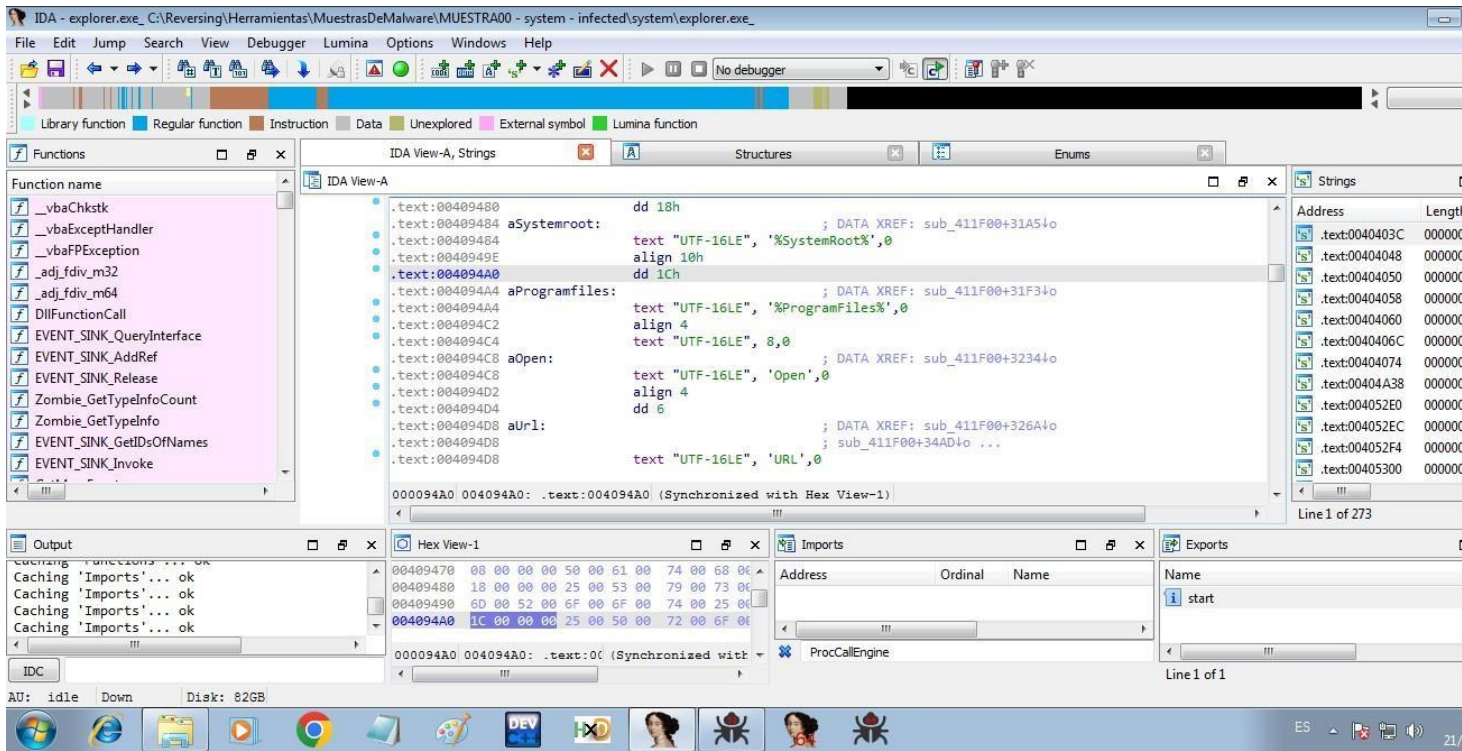
URLDownloadToFile: Downloads bits from the Internet and saves them to a file.

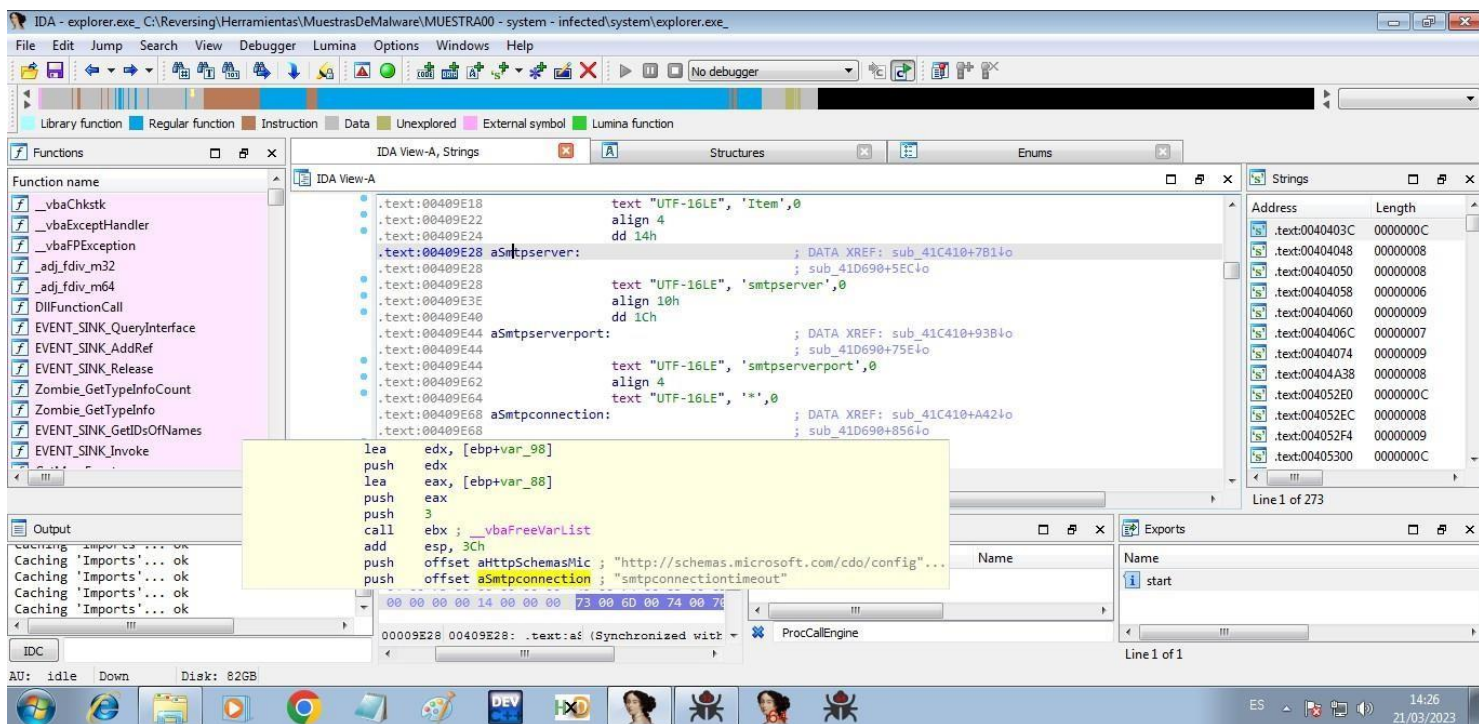
DeleteUrlCacheEntryA: Deletes the file associated with the source name from the cache, if the file exists.

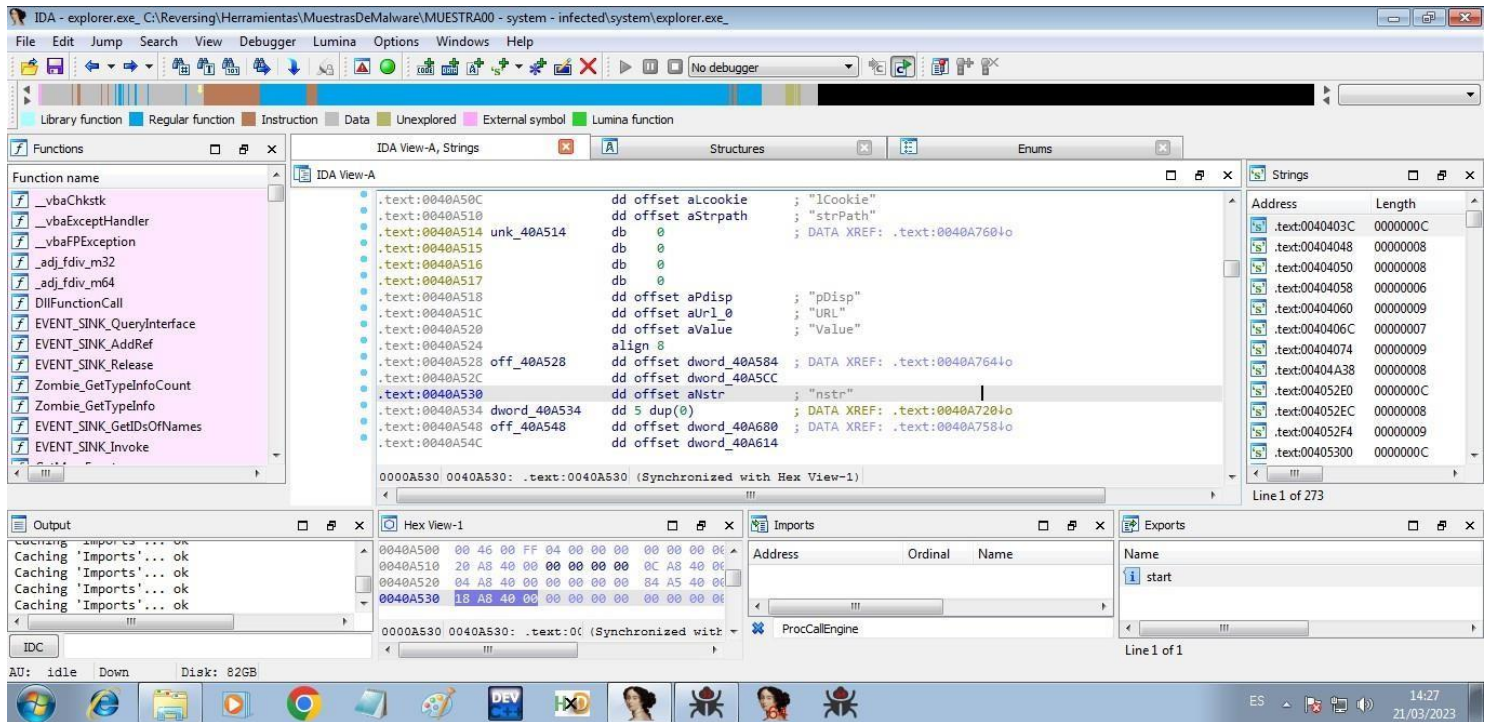
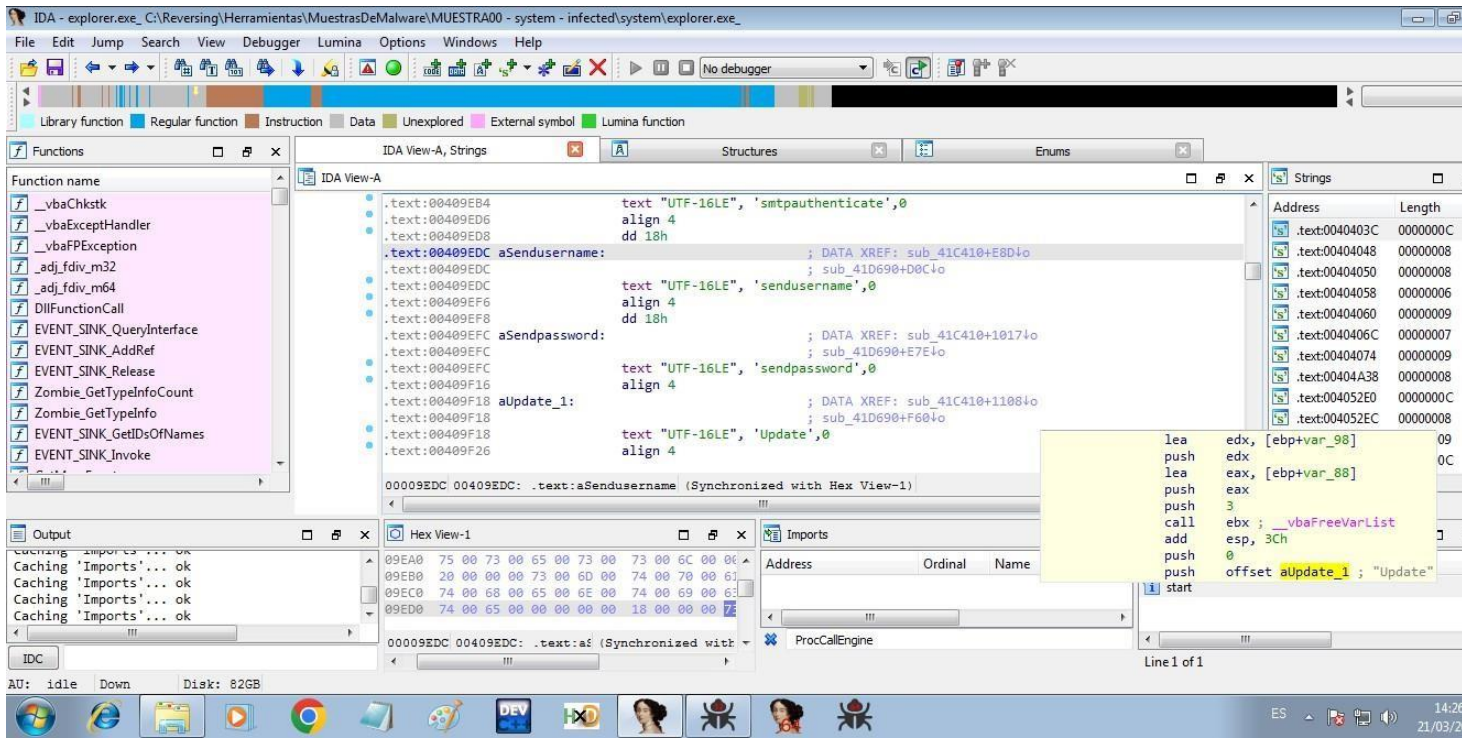
InternetCloseHandle: Closes a single Internet handle.

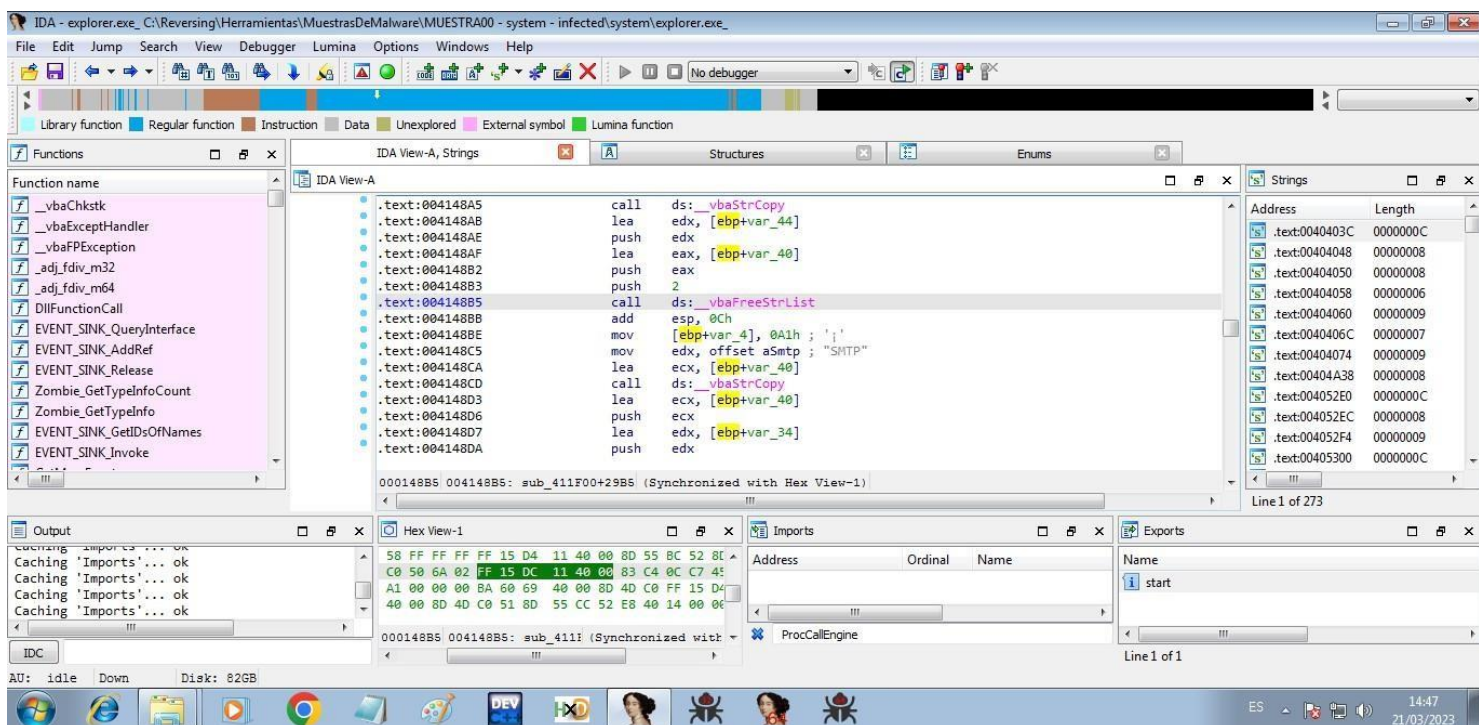
GetSetting: Returns a key setting value from an application's entry in the Windows Registry or information (on Macintosh) in the application's initialization file.

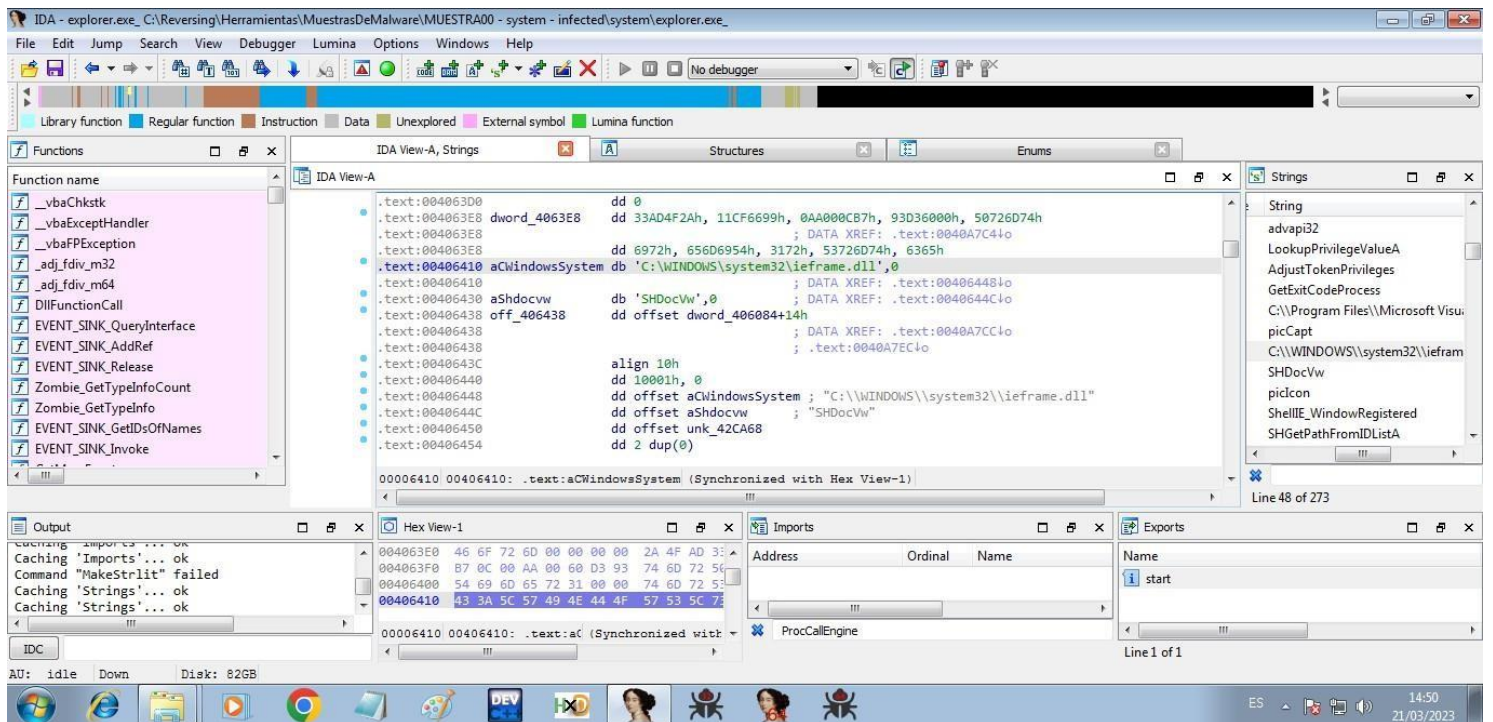
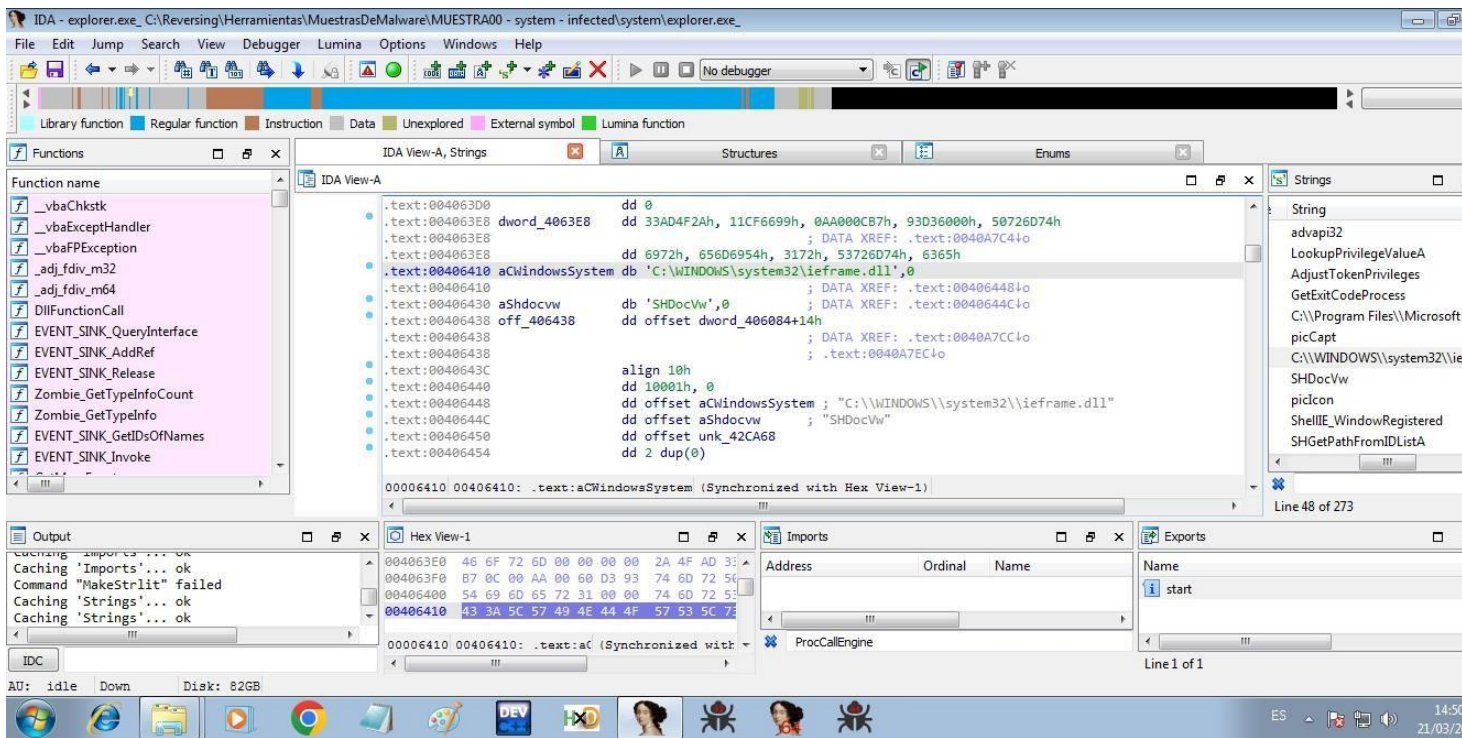


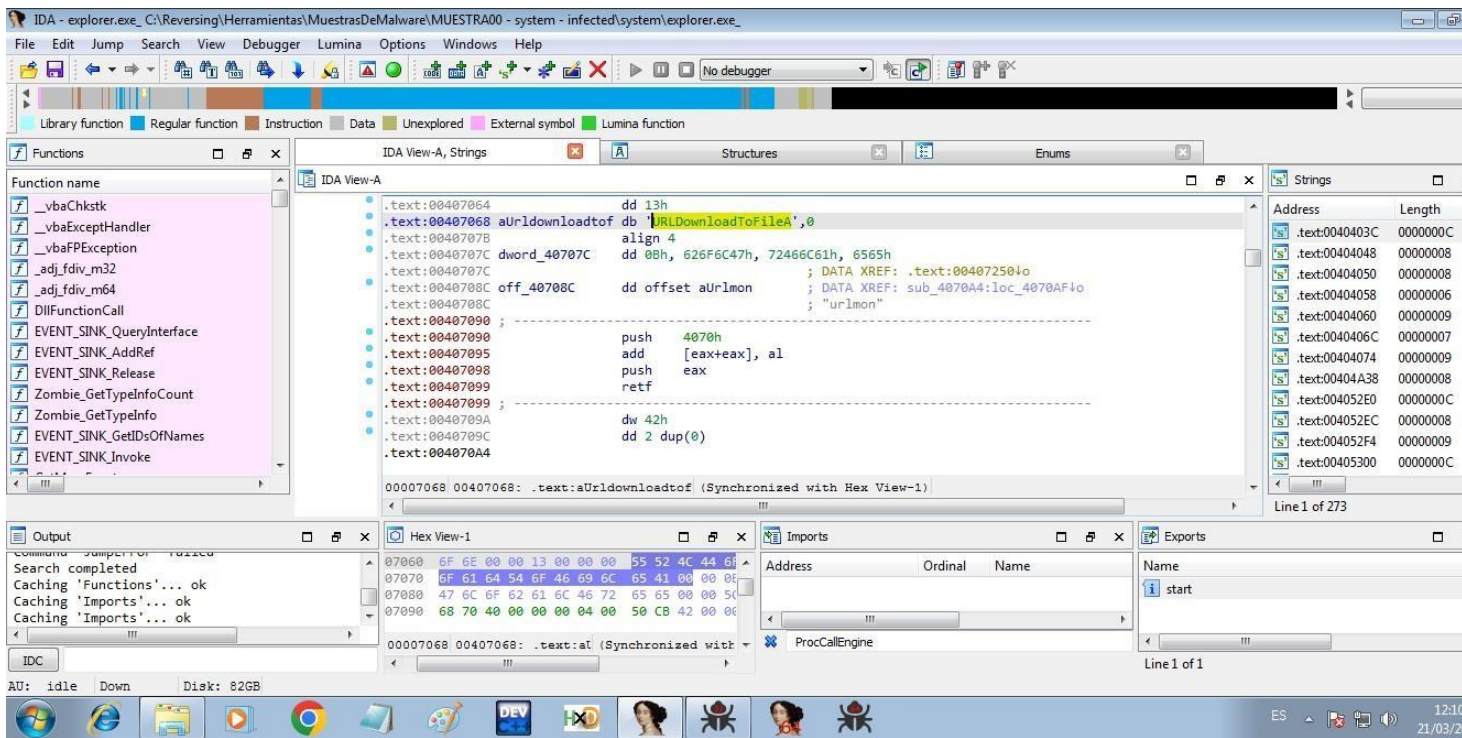






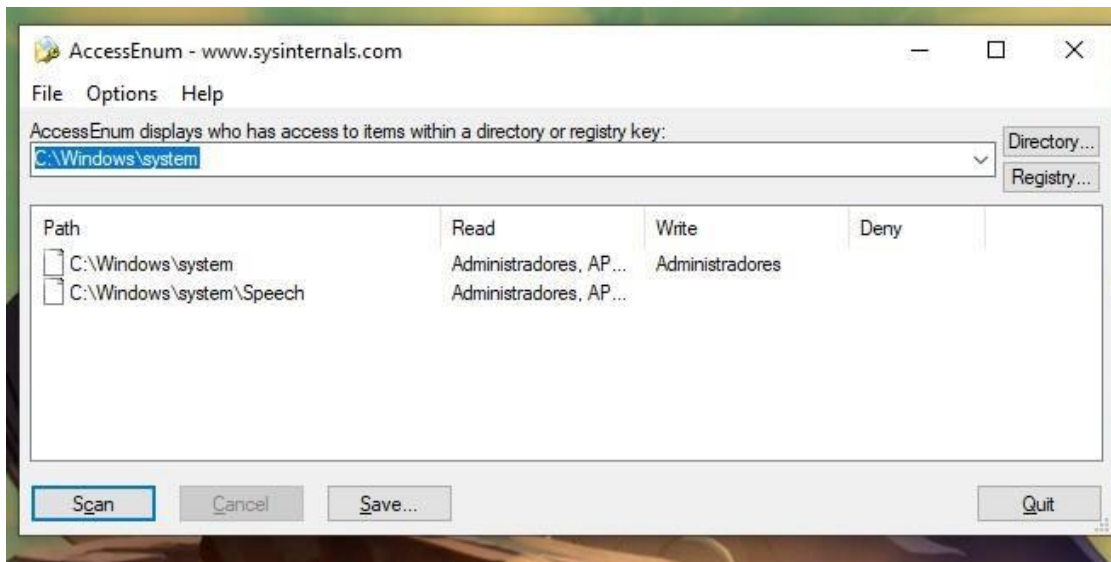




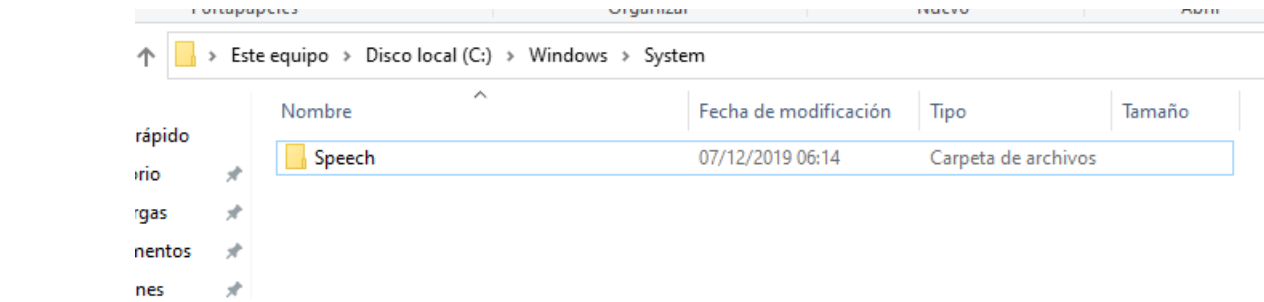


At this point, I decided to execute the sample since I couldn't find anything else to do. Before infection:

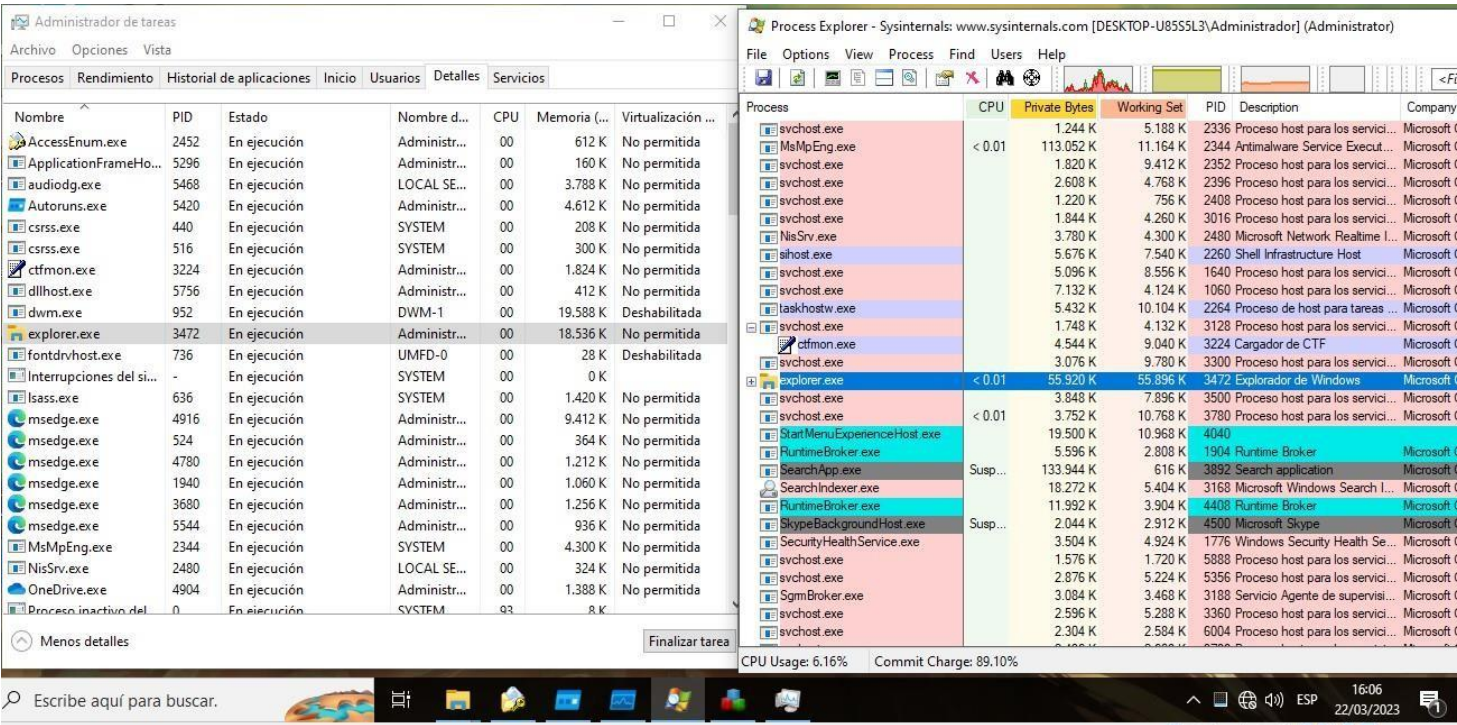
Here, using AccessEnum, I can see that this folder (which appears in previous screenshots involving strings and operations) has these permissions – (It is worth noting that after the infection, this remains unchanged).



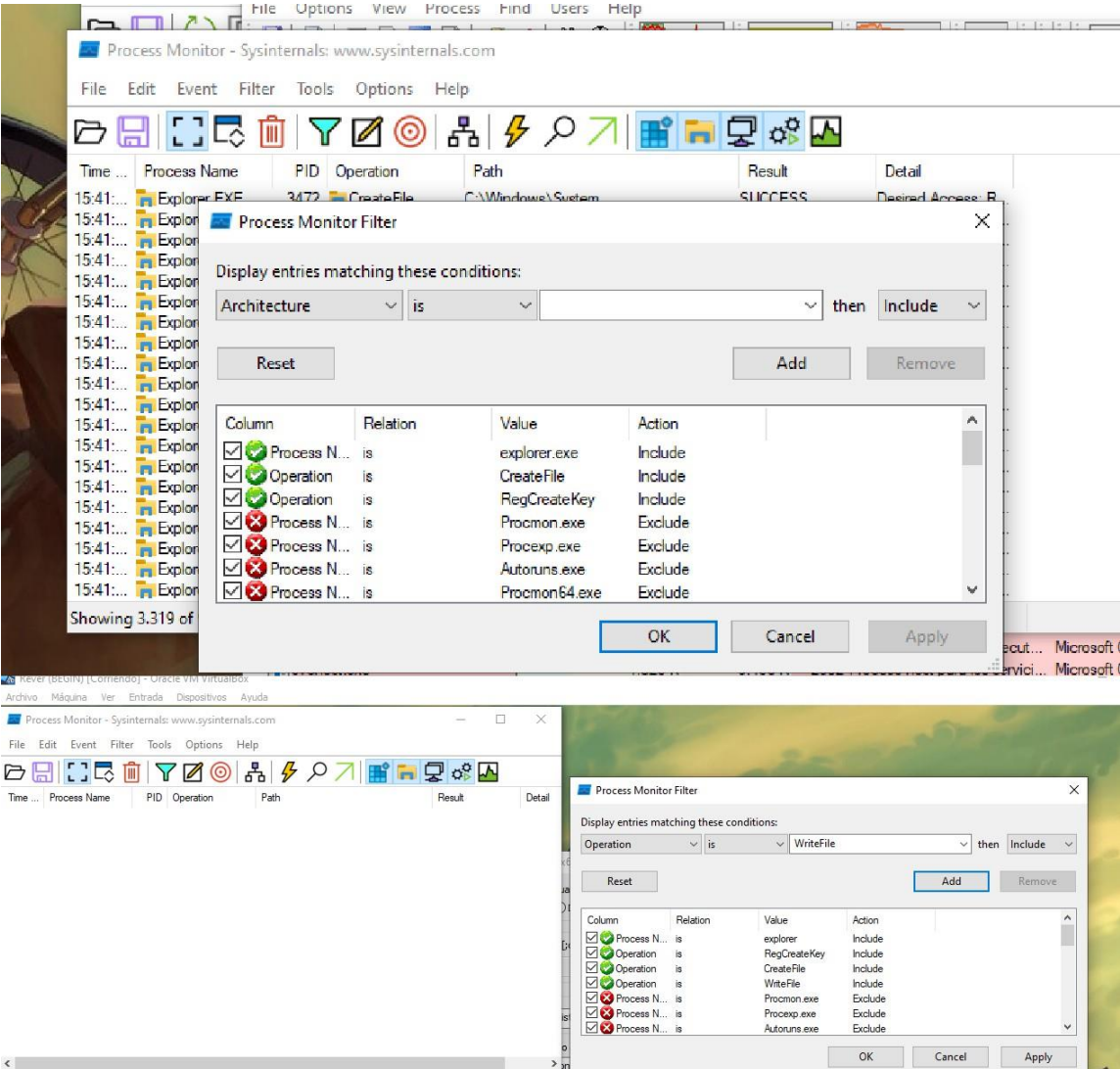
This folder now only contains the Speech folder.



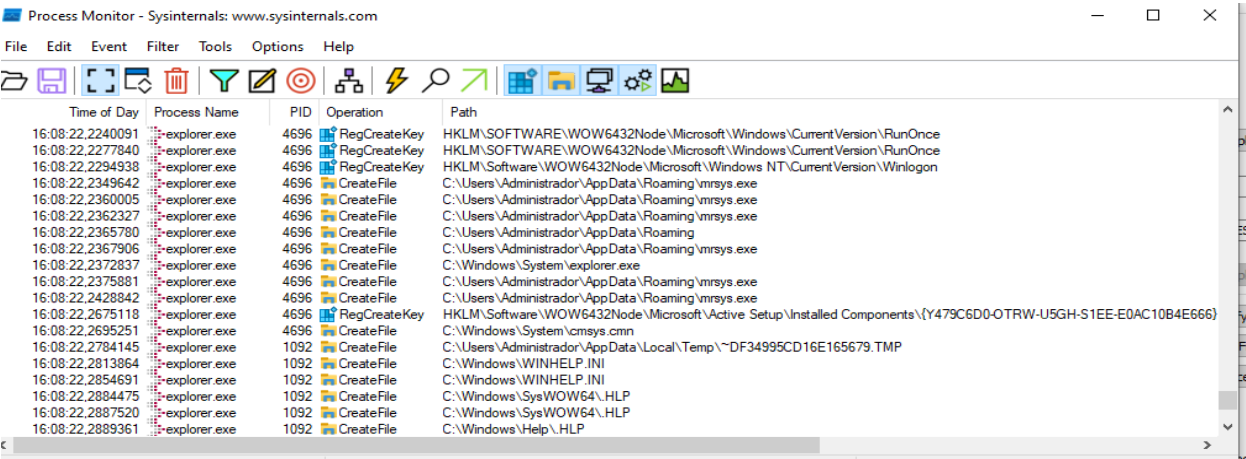
Process Monitor and Task Manager only show a single explorer.exe process.



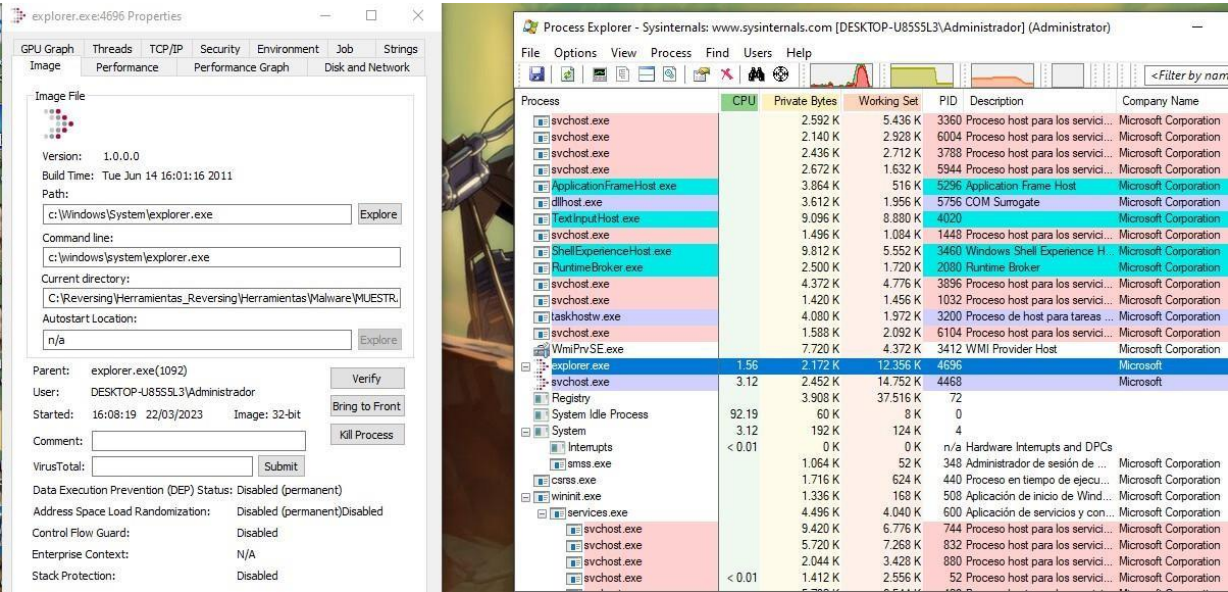
In Process Monitor, I use three filters to better visualize the information.



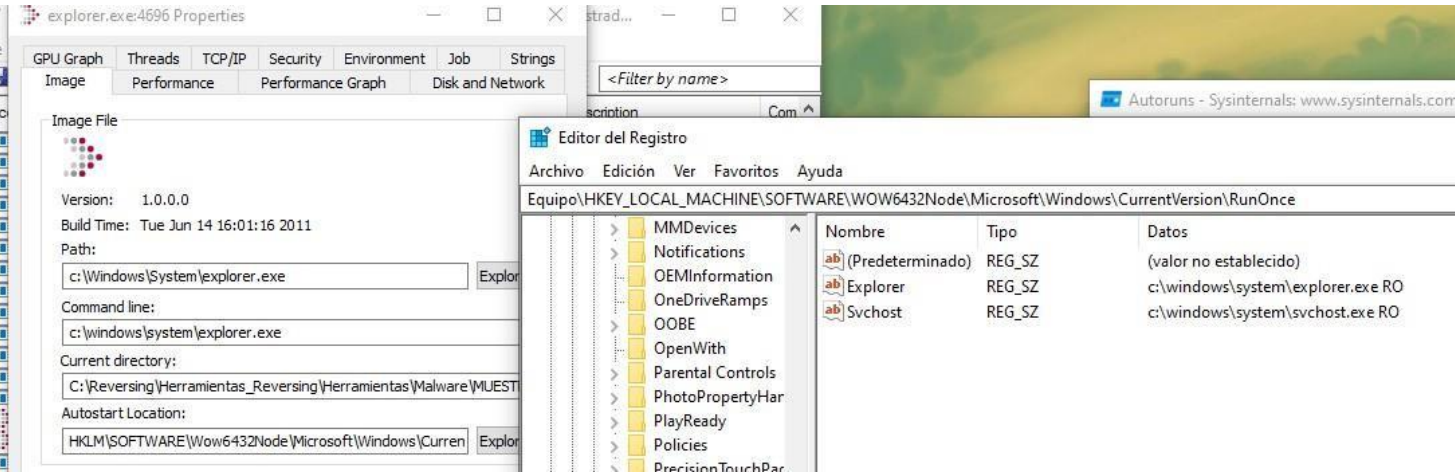
Here, I found registry entries that were added and were not there before, which leads me to believe that it achieves persistence.



Here, I can confirm that it is indeed residing in the C:\Windows\System\ path.



We can also see that it indeed creates two registry keys. Additionally, if you try to terminate the explorer.exe process featuring that icon, it restarts automatically (as a result of these registry keys).



Finally, to verify, I navigate to the path. Since I cannot find them, I open a command prompt and run `attrib -h -r -s /s /d`, which reveals those same three files responsible for the infection.

Acceso rápido

Portapapeles

Organizar

Nuevo

Abrir

Este equipo > Disco local (C:) > Windows > System >

Acceso rápido

Escritorio

Descargas

Documentos

Imágenes

Música

Videos

OneDrive

Este equipo

Red

Nombre	Fecha de modificación	Tipo	Tamaño
Speech	07/12/2019 06:14	Carpeta de archivos	
cmsys.cmn	22/03/2023 16:08	Archivo CMN	0 KB
explorer.exe	22/03/2023 16:08	Aplicación	275 KB
spoolsv.exe	22/03/2023 16:08	Aplicación	275 KB
svchost.exe	22/03/2023 16:08	Aplicación	275 KB

Administrador: Símbolo del sistema

Microsoft Windows [Versión 10.0.19045.2006]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Administrador>cd c:\Windows\System

c:\Windows\System>attrib -h -r -s /s /d
Acceso denegado: C:\Windows\System\Speech\speech-synthesis.xsd
Acceso denegado: C:\Windows\System\Speech\synthesis-core.xsd
Acceso denegado: C:\Windows\System\Speech\xml.xsd
Acceso denegado: C:\Windows\System\Speech

c:\Windows\System>